

Lab 2 – Dart Essentials Practice Lab

1. Objective

This lab helps students build a strong foundation in Dart by practicing core language features including program structure, data types, variables, collections, control flow, functions, OOP basics, null safety, async/await, and streams.

All exercises can run directly on **DartPad**, **Android Studio**, or **VS Code**.

2. Requirements

Complete **five exercises** covering the fundamental concepts of Dart:

1. **Basic Syntax & Data Types** – practice printing output and using core types.
2. **Collections & Operators** – manipulate lists, sets, and maps using Dart operators.
3. **Control Flow & Functions** – implement logic using if/else, switch, loops, and functions.
4. **Intro OOP** – create classes, constructors, inheritance, and method overriding.
5. **Async & Null Safety** – work with async/await, Futures, and null-safety operators.

Each task must:

- Compile and run successfully.
- Print results to the console.
- Contain comments explaining the code.

3. Guided Steps

Exercise 1 – Basic Syntax & Data Types

Goal: Practice program structure and variable declarations.

Steps

1. Create a `main()` function.
2. Declare variables using: `int`, `double`, `String`, `bool`.
3. Use `print()` and **string interpolation** (`$var`, `${expr}`) to show values.

Exercise 2 – Collections & Operators

Goal: Work with `List`, `Set`, `Map` and operators (`+`, `-`, `==`, `&&`, `?` :).

Steps

1. Create a `List` of integers.
2. Use arithmetic & comparison operators.
3. Create a `Set` (unique values) and a `Map` (key-value).
4. Use indexing, `add()`, `remove()`, and map access.

Exercise 3 – Control Flow & Functions

Goal: Apply `if/else`, `switch`, loops, and functions.

Steps

1. Write an `if/else` block to check score.
2. Write a `switch case` for day of week.
3. Loop through a collection using `for`, `for-in`, and `forEach()`.
4. Create a function using normal and arrow syntax.

Exercise 4 – Intro to OOP

Goal: Practice classes, objects, constructors, inheritance, and overriding.

Steps

1. Create a class `Car` with one property and a method.

2. Create a named constructor.
3. Create a subclass `ElectricCar` that overrides a method.
4. Instantiate objects and print results.

Exercise 5 – Async, Future, Null Safety & Streams

Goal: Work with Dart's asynchronous features.

Steps

1. Create an async function using `Future + await`.
2. Use `Future.delayed()` to simulate loading.
3. Practice null-safety operators (`?`, `??`, `!`).
4. Create a simple Stream of integers and listen to values.

4. Expected Results

- Students execute all exercises without errors.
- Output shows correct values, loop counts, OOP behavior, and async sequencing.
- Students understand Dart essentials needed for Module 3 onward.

5. Submission

- **Deliverable:** A single Dart file containing all exercises, or a zipped project.
- **Evaluation Criteria:**
 - Correctness (40%)
 - Use of Dart features (25%)
 - Output accuracy (20%)
 - Code readability & comments (15%)